Aşama F1: getirme adresi

Bu çevrimde getirilecek buyruğun adresi L3 program belleğine verilir ve bir sonraki çevrimin getirme adresi seçilir. Dallanma D2'de karar verdiği için dal hedefi bir sonraki çevrimin F1'ine yetişir; aradaki 3 buyruk gecikme yuvasıdır.

boru hattı yazmacı (FF) · mimari durum (FF) · kombinasyonel blok · BRAM · ALU · önerilen arayüz (mevcut modülden farklı) · saat kenarı · ▶ girdi · ◀ çıktı · sağdaki sayı = bit

Dışarıdan gelen sinyaller

sinyal	bit	kaynak
getir_izin	1	görev denetçisi
gorev_basla	1	görev denetçisi
mvect_pc	12	cla_mvect_regs.target_pc_out[11:0]
dal_alindi	1	D2 · dal birimi
dal_hedef	12	D2 · dal birimi
en_getir_pc, en_F1F2, balon_F1F2	1 · 1 · 1	boru hattı denetimi

Dışarı giden sinyaller

sinyal	bit	hedef
l3_adr	11	L3 program BRAM (F2), en_F1F2 ile kenarda içine alınır

getir_pc · 12 FF

Getirme sayacı (MPC değil). F1'deki buyruğun adresi.

▶ girdi

sonraki_pc	12
en_getir_pc	1
clk · rst_n	

◀ çıktı

getir_pc	12

Nasıl çalışır

```
saat kenarında:
  en_getir_pc ise
    getir_pc <= sonraki_pc
  değilse korunur
```

F1 · getirme adresi

Sonraki getirme adresi

L3'e bu çevrimin adresini verir, getirme sayacının bir sonraki değerini seçer.

▶ girdi

getir_pc	12
getir_izin	1
gorev_basla	1
mvect_pc	12
dal_alindi	1
dal_hedef	12

◀ çıktı

l3_adr	11
f1_gecerli	1
f1_pc	12
sonraki_pc	12

Nasıl çalışır

```
l3_adr     = getir_pc[11:1]   // 32 bitlik satır
f1_pc      = getir_pc
f1_gecerli = getir_izin
sonraki_pc = gorev_basla ? mvect_pc
           : dal_alindi  ? dal_hedef
           : getir_izin  ? getir_pc + 2
           :               getir_pc
// 12 bit, taşma atılır; buyruk = 2 kelime
// öncelik: görev başı > dal > sıradaki
```

F1/F2 · 13 FF

F2'deki buyruğun adresi ve geçerliliği. L3 BRAM'in adres yazmacı da bu kenardadır.

▶ girdi

f1_gecerli	1
f1_pc	12
en_F1F2	1
balon_F1F2	1
clk · rst_n	

◀ çıktı

geçerli	1
pc	12

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
```

Aşama F2: buyruk kelimesinin okunması

F1'de verilen adres L3 BRAM'in adres yazmacına girmiştir; F2 boyunca 32 bitlik buyruk kelimesi BRAM çıkışında durur ve çevrimin sonunda F2/D1'e yazılır. pc ve geçerli bit değişmeden geçer.

boru hattı yazmacı (FF) · mimari durum (FF) · kombinasyonel blok · BRAM · ALU · önerilen arayüz (mevcut modülden farklı) · saat kenarı · ▶ girdi · ◀ çıktı · sağdaki sayı = bit

Dışarıdan gelen sinyaller

sinyal	bit	kaynak
en_F1F2, en_F2D1, balon_F2D1	1 · 1 · 1	boru hattı denetimi

F1/F2 · 13 FF

▶ girdi

f1_gecerli	1
f1_pc	12
en_F1F2	1
balon_F1F2	1
clk · rst_n	

◀ çıktı

geçerli	1
pc	12

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
```

F2 · kelime okuma

L3 program BRAM

Program belleği (2K x 32 = 4K x 16). Adresteki buyruk kelimesini bir çevrim sonra verir.

▶ girdi		◀ çıktı	
l3_adr	11	kelime	32
en (= en_F1F2)	1		
clk			

Nasıl çalışır

```
saat kenarında (F1 sonu):
  if (en) dout <= mem[l3_adr]
en = 0 iken dout korunur (bekleme)
satır = {MSW, LSW}  (casm2hex, 32 bit satır)
içerik: $readmemh ile yüklenir
```

Geçiş

pc ve geçerli bit F2'de değişmez.

▶ girdi		◀ çıktı	
geçerli	1	geçerli	1
pc	12	pc	12

Nasıl çalışır

```
F2/D1.geçerli = F1/F2.geçerli
F2/D1.pc      = F1/F2.pc
```

F2/D1 · 45 FF

▶ girdi

geçerli	1
pc	12
kelime	32
en_F2D1	1
balon_F2D1	1
clk · rst_n	

◀ çıktı

geçerli	1
pc	12
kelime	32

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
```

Aşama D1: buyruğun çözülmesi

cla_decoder 32 bitlik kelimeyi tamamen kombinasyonel olarak çözer: hangi buyruk olduğu (op) ve alanlarının değerleri (alan). Sonuç D1/D2'ye yazılır; kontrol paketi bir sonraki aşamada (D2) üretilir.

boru hattı yazmacı (FF) · mimari durum (FF) · kombinasyonel blok · BRAM · ALU · önerilen arayüz (mevcut modülden farklı) · saat kenarı · ▶ girdi · ◀ çıktı · sağdaki sayı = bit

Dışarıdan gelen sinyaller

sinyal	bit	kaynak
en_F2D1, en_D1D2, balon_D1D2	1 · 1 · 1	boru hattı denetimi

F2/D1 · 45 FF

▶ girdi

geçerli	1
pc	12
kelime	32
en_F2D1	1
balon_F2D1	1
clk · rst_n	

◀ çıktı

geçerli	1
pc	12
kelime	32

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
```

D1 · çözme

cla_decoder

Buyruk kelimesini tanır ve alanlarını sabit yerlere toplar.

▶ girdi

buyruk (= kelime)	32

◀ çıktı

op	7
gecersiz	1
kip_gecersiz	1
alan	43
ra · rb · rc	2 · 2 · 2
rd · re · rf	2 · 2 · 2
mar_x	1
addr_kip	4
cndf	4
shift	6
m	16

Nasıl çalışır

```
1) unique casez (buyruk): 78 desen (cla_type0.yaml)
   MAR0/MAR1 çiftleri tek kimlik → 74 op
   eşleşme yoksa op = OP_GECERSIZ, gecersiz = 1
2) alan: önce varsayılan konumlar
   ra[1:0] rb[3:2] rc[5:4] rd[7:6] re[9:8] rf[11:10]
   addr_kip = [19:16]  m = [15:0]  cndf = UNCF
   shift = [6:2] + 1   (1 … 32)
   sonra gruba göre ezilir, ör. paralel FP:
   ra[21:20] rd[23:22] re[25:24] rf[27:26]
3) kip_gecersiz = bellek buyruğu ∧
   addr_kip ∉ {0000, 0001, 0010}
örnek: 0x73C08800 → OP_MMOV32_MR_MEM
   ra = 0, cndf = 1111, kip = 0000, m = 0x8800
```

Geçiş

pc, kelime ve geçerli bit değişmeden geçer.

▶ girdi		◀ çıktı	
geçerli	1	geçerli	1
pc	12	pc	12
kelime	32	kelime	32

Nasıl çalışır

```
D1/D2'ye aynen yazılır (kelime yalnız iz için)
```

D1/D2 · 96 FF

▶ girdi

geçerli	1
pc	12
kelime	32
op	7
alan	43
kip_gecersiz	1
en_D1D2	1
balon_D1D2	1
clk · rst_n	

◀ çıktı

geçerli	1
pc	12
kelime *	32
op	7
alan	43
kip_gecersiz	1

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
* yalnız iz için
```

Aşama D2: kontrol paketi, etkin adres, MAR artışı ve dal kararı

Kontrol paketi üretilir, bellek adresi hesaplanır, MAR artırılır, dal kararı verilir; MSTOP ve geçersiz kod görev denetçisine bildirilir. MAR, RPC ve dal güncellemeleri yalnız D2 ilerlerken (d2_ilerle) yapılır; beklemede hiçbiri olmaz.

boru hattı yazmacı (FF) · mimari durum (FF) · kombinasyonel blok · BRAM · ALU · önerilen arayüz (mevcut modülden farklı) · saat kenarı · ▶ girdi · ◀ çıktı · sağdaki sayı = bit

Dışarıdan gelen sinyaller

sinyal	bit	kaynak
MAR0, MAR1	16 · 16	MAR yazmaçları
RPC	12	MSTF
gorunen_5	5	W · görünen MSTF
dur_exe	1	W · dur_exe birimi
dur_r1	1	R1 · dur_r1 birimi
en_D1D2, en_D2R1, balon_D2R1	1 · 1 · 1	boru hattı denetimi

Dışarı giden sinyaller

sinyal	bit	hedef
dal_alindi, dal_hedef	1 · 12	F1 · sonraki adres
mar_artis_en/_sec/_deger	1 · 1 · 16	MAR yazmaçları
rpc_en, rpc_deger	1 · 12	MSTF
d2_gecerli, d2_mstop, d2_gecersiz	1 · 1 · 1	görev denetçisi
mpc_en, mpc_deger	1 · 12	MPC

D1/D2 · 96 FF

▶ girdi

geçerli	1
pc	12
kelime	32
op	7
alan	43
kip_gecersiz	1
en_D1D2	1
balon_D1D2	1
clk · rst_n	

◀ çıktı

geçerli	1
pc	12
kelime *	32
op	7
alan	43
kip_gecersiz	1

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
* yalnız iz için
```

D2 · kontrol, adres, dal

cla_ctrl_gen

op ve alanlardan buyruğun bütün statik kararlarını (kontrol paketi) üretir. Kombinasyonel.

▶ girdi		◀ çıktı	
op	7	k (kontrol_t)	102
alan	43		
kip_gecersiz	1		

Nasıl çalışır

```
k = '0;  k.kosul = alan.cndf
k.alu.karsilastirma_cndf = UNCF
case (op): her buyruk yalnız kendi alanlarını kurar
  ör. OP_MADDF32_RRR:
     a_mr = rb, b_mr = rc, okunan_mr = (1<<rb)|(1<<rc)
     alu.topla_kullan = 1, alu.topla_hedef = ra
     alu.mux[1] = 00 (a), alu.mux[2] = 01 (b)
  ör. OP_MMOV32_MEM_MR:
     bellek_adres = 1, bellek_yaz = 11,
     yaz_veri = VERI_MR, veri_mr = ra
op = OP_GECERSIZ ∨ kip_gecersiz:
     k = 0, k.gecersiz = 1
```

Etkin adres ve MAR artışı

Bellek buyruğunun adresini kipe göre seçer; MAR kipinde MAR'a ofseti ekler.

▶ girdi		◀ çıktı	
addr_kip	4	ea	16
m	16	mar_artis_en	1
MAR0	16	mar_artis_sec	1
MAR1	16	mar_artis_deger	16
bellek_adres (k)	1		
d2_ilerle	1		

Nasıl çalışır

```
ea = (addr_kip == 0000) ? m
   : (addr_kip == 0001) ? MAR0
   :                      MAR1   // 0010
mar_artis_en    = d2_ilerle ∧ bellek_adres ∧
                  addr_kip ∈ {0001, 0010}
mar_artis_sec   = addr_kip[1]   // 0: MAR0, 1: MAR1
mar_artis_deger = ea + m        // 16 bit
koşullu MMOV32'de de artar (koşul E'de)
```

D2 olayları

D2'nin bu çevrimde işlenip işlenmediğini belirler; görev denetçisine ve MPC'ye bildirir.

▶ girdi		◀ çıktı	
geçerli	1	d2_ilerle	1
dur_exe	1	d2_gecerli	1
dur_r1	1	d2_mstop	1
gecersiz (k)	1	d2_gecersiz	1
mstop (k)	1	mpc_en	1
pc	12	mpc_deger	12

Nasıl çalışır

```
d2_ilerle   = geçerli ∧ ¬dur_exe ∧ ¬dur_r1
d2_gecerli  = d2_ilerle   // denetçi: D2 işi bir kez
d2_mstop    = k.mstop
d2_gecersiz = k.gecersiz
mpc_en      = geçerli     // MPC = D2'deki buyruk,
mpc_deger   = pc          // beklemede de
```

Dal birimi

MBCNDD, MCCNDD ve MRCNDD için koşulu değerlendirir, hedefi ve dönüş adresini üretir.

▶ girdi		◀ çıktı	
dal (k)	2	dal_alindi	1
kosul (k)	4	dal_hedef	12
pc	12	rpc_en	1
m	16	rpc_deger	12
RPC	12		
gorunen_5	5		
d2_ilerle	1		

Nasıl çalışır

```
tuttu = CNDF(kosul, gorunen_5)   // koşul tablosu
dal_alindi = d2_ilerle ∧ (dal ≠ YOK) ∧ tuttu
dal_hedef  = (dal == MRCNDD) ? RPC
           : (pc + m)[11:0]   // taban = dalın adresi
rpc_en     = d2_ilerle ∧ (dal == MCCNDD) ∧ tuttu
rpc_deger  = pc + 8      // dal + 3 gecikme yuvası
gecikme yuvaları silinmez (F1–D1 sürer)
```

D2/R1 · 185 FF

R1'deki buyruğun paketi ve adresi.

▶ girdi

geçerli	1
pc	12
kelime	32
k	102
ea	16
m	16
shift	6
en_D2R1	1
balon_D2R1	1
clk · rst_n	

◀ çıktı

geçerli	1
pc	12
kelime *	32
k (kontrol_t)	102
ea	16
m	16
shift	6
toplam	185

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
* yalnız iz için
MSTOP D2'deyse balon_D2R1 = 1
```

Aşama R1: okuma adresi ve bellek portu

Okunacak veri bloğu ve blok içi adres çıkarılır, BRAM'e verilir; adres R1 sonunda BRAM'e girer, veri R2'de çıkar. Aynı çevrimde W aynı bloğa yazıyorsa R1 bir çevrim bekler (dur_r1); port o çevrimde W'nin yazmasına verilir.

boru hattı yazmacı (FF) · mimari durum (FF) · kombinasyonel blok · BRAM · ALU · önerilen arayüz (mevcut modülden farklı) · saat kenarı · ▶ girdi · ◀ çıktı · sağdaki sayı = bit

Dışarıdan gelen sinyaller

sinyal	bit	kaynak
w_gecerli, w_bellek_yaz	1 · 2	E/W yazmacı
w_blok, w_adr, w_wr_en, w_din	2 · 10 · 2 · 32	W · RAM yazma
dur_exe	1	W · dur_exe birimi
en_D2R1, en_R1R2, balon_R1R2	1 · 1 · 1	boru hattı denetimi

Dışarı giden sinyaller

sinyal	bit	hedef
L1, L2, C2M, M2C portları	bkz. port seçici	veri BRAM'leri
dur_r1	1	boru hattı denetimi, D2, R1

D2/R1 · 185 FF

▶ girdi

D2'den alanlar	185
en_D2R1	1
balon_D2R1	1
clk · rst_n	

◀ çıktı

geçerli	1
pc	12
kelime *	32
k (kontrol_t)	102
ea	16
m	16
shift	6
toplam	185

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
```

R1 · okuma adresi

Okuma adresi

ea'dan bloğu ve blok içi adresi çıkarır, okuma iznini üretir.

▶ girdi

ea	16
bellek_oku (k)	2
geçerli	1
dur_exe	1
dur_r1	1

◀ çıktı

r1_blok	2
r1_adr	10
r1_rd_en	2

Nasıl çalışır

```
r1_blok = 0x8800–0x8BFF: L1 (00)
          0x8C00–0x8FFF: L2 (01)
          0x1480–0x14FF: C2M (10)
          0x1500–0x157F: M2C (11)
r1_adr  = ea − taban
          32 bit erişimde r1_adr[0] = 0 (çifte hizala)
r1_rd_en = (geçerli ∧ ¬dur_exe ∧ ¬dur_r1)
           ? bellek_oku : 00
```

dur_r1 birimi

W'deki yazma ile R1'deki okumanın aynı bloğa düşmesini yakalar.

▶ girdi

r1_gecerli	1
bellek_oku	2
r1_blok	2
w_gecerli	1
w_bellek_yaz	2
w_blok	2

◀ çıktı

dur_r1	1

Nasıl çalışır

```
dur_r1 = r1_gecerli ∧ (bellek_oku ≠ 00) ∧
         w_gecerli ∧ (w_bellek_yaz ≠ 00) ∧
         (r1_blok == w_blok) ∧ (r1_blok ≠ M2C)
kombinasyonel; dur_exe ile birlikteyse dur_exe baskın
```

RAM port seçici

Her bloğun tek portunu ya W'nin yazmasına ya R1'in okumasına verir.

▶ girdi

r1_blok	2
r1_adr	10
r1_rd_en	2
w_blok	2
w_adr	10
w_wr_en	2
w_din	32

◀ çıktı

L1: addr · rd_en · wr_en · din	10 · 2 · 2 · 32
L2: addr · rd_en · wr_en · din	10 · 2 · 2 · 32
C2M: addr · rd_en · wr_en · din	7 · 2 · 2 · 32
M2C: addr · rd_en	7 · 2

Nasıl çalışır

```
her blok b için:
  W bu bloğa yazıyorsa (w_wr_en ≠ 0 ∧ w_blok == b):
    addr = w_adr, wr_en = w_wr_en, din = w_din,
    rd_en = 00
  değilse:
    addr = r1_adr, wr_en = 00,
    rd_en = (r1_blok == b) ? r1_rd_en : 00
dur_r1 aynı bloktaki çakışmayı zaten önler
```

R1/R2 · 185 FF

R2'deki buyruk. BRAM'lerin adres/veri yazmacı da bu kenardadır.

▶ girdi

D2/R1 alanları	185
en_R1R2	1
balon_R1R2	1
clk · rst_n	

◀ çıktı

geçerli	1
pc	12
kelime *	32
k (kontrol_t)	102
ea	16
m	16
shift	6
toplam	185

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
dur_r1 ise balon_R1R2 = 1
```

Aşama R2: okunan verinin seçilmesi

R1 sonunda BRAM'lere giren adreslerin verisi R2 boyunca BRAM çıkışlarında durur. Buyruğun okuduğu bloğun çıkışı seçilir, 16 bitlik erişimde üst yarı sıfırlanır ve R2/E'ye yazılır.

boru hattı yazmacı (FF) · mimari durum (FF) · kombinasyonel blok · BRAM · ALU · önerilen arayüz (mevcut modülden farklı) · saat kenarı · ▶ girdi · ◀ çıktı · sağdaki sayı = bit

Dışarıdan gelen sinyaller

sinyal	bit	kaynak
C2M, M2C dout	32 · 32	köprü (cla_kopru_cekirdek)
en_R1R2, en_R2E	1 · 1	boru hattı denetimi

R1/R2 · 185 FF

▶ girdi

D2/R1 alanları	185
en_R1R2	1
balon_R1R2	1
clk · rst_n	

◀ çıktı

geçerli	1
pc	12
kelime *	32
k (kontrol_t)	102
ea	16
m	16
shift	6
toplam	185

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
```

R2 · okunan veri

Veri BRAM (L1, L2: her biri)

1K × 16 veri bloğu, tek port. 32 bit erişimde adr ve adr + 1 birlikte.

▶ girdi		◀ çıktı	
addr	10	dout	32
wr_en	2		
rd_en	2		
din	32		
clk			

Nasıl çalışır

```
saat kenarında:
  wr_en[0]: mem[addr]     <= din[15:0]
  wr_en[1]: mem[addr + 1] <= din[31:16]
  rd_en ≠ 0: dout <= {mem[addr + 1], mem[addr]}
  rd_en = 0: dout korunur
(cla_data_ram şu an rd_en = 0'da dout'u 0'lıyor)
```

Okunan seçici

Buyruğun okuduğu bloğun çıkışını seçer ve genişliğe göre düzenler.

▶ girdi		◀ çıktı	
dout_L1	32	okunan	32
dout_L2	32		
dout_C2M	32		
dout_M2C	32		
ea	16		
bellek_oku (k)	2		

Nasıl çalışır

```
blok = çöz(ea)          // R1 ile aynı kural
okunan = dout_blok
bellek_oku == 01 (16 bit): okunan[31:16] = 0
C2M, M2C: köprüdeki mesaj RAM'lerinin CLA portu
```

R2/E · 217 FF

E'deki buyruk: paket, adres, anlık değerler ve okunan veri.

▶ girdi

R1/R2 alanları	185
okunan	32
en_R2E	1
balon_R2E	1
clk · rst_n	

◀ çıktı

geçerli	1
pc	12
kelime *	32
k (kontrol_t)	102
ea	16
m	16
shift	6
okunan	32
toplam	217

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
```

Aşama E: yürütme

İşlenenler MR'lerden ve anlık değerlerden toplanır, ALU çalışır. Kombinasyonel sonuçlar (tamsayı, kestirim, karşılaştırma, dönüşüm, bellekten yükleme) çevrimin sonunda MR'ye, MAR'a ve MSTF'ye yazılır. Toplama ve çarpma K1'i yapar, ara sonucu K yazmacına bırakır.

boru hattı yazmacı (FF) · mimari durum (FF) · kombinasyonel blok · BRAM · ALU · önerilen arayüz (mevcut modülden farklı) · saat kenarı · ▶ girdi · ◀ çıktı · sağdaki sayı = bit

Dışarıdan gelen sinyaller

sinyal	bit	kaynak
MR0–MR3	4 × 32	MR yazmaçları
MAR0, MAR1	16 · 16	MAR yazmaçları
RND32	1	MSTF
gorunen_5, gorunen_32	5 · 32	W · görünen MSTF
dur_exe	1	W · dur_exe birimi
dondur	1	görev denetçisi
en_R2E, en_EW, balon_EW	1 · 1 · 1	boru hattı denetimi

Dışarı giden sinyaller

sinyal	bit	hedef
e_yaz1, exe_hedef, e_veri1	1 · 2 · 32	W · MR yazma birimi
e_yaz2, exe_hedef_2, e_veri2	1 · 2 · 32	W · MR yazma birimi
mar_veri, mar_sec, mar_en	16 · 1 · 1	MAR yazmaçları
e_mstf_veri, e_mstf_maske	19 · 19	MSTF
okunan_mr	4	W · dur_exe birimi

R2/E · 217 FF

▶ girdi

R1/R2 alanları	185
okunan	32
en_R2E	1
balon_R2E	1
clk · rst_n	

◀ çıktı

geçerli	1
pc *	12
kelime *	32
ea	16
m	16
shift	6
okunan	32
kontrol_t	102
kosul · bellek_oku	4 · 2
alu	42
a_mr · b_mr · c_mr	2 · 2 · 2
d_kaynak · d_mr	2 · 2
okunan_mr	4
exe_yaz · exe_hedef	1 · 2
exe_kaynak · exe_kaynak_mr	3 · 2
exe_yaz_2 · exe_hedef_2	1 · 2
exe_kaynak_mr_2	2
exe_kosullu	1
mar_yukle · mar_hedef	1 · 1
mar_kaynak	2
zn_kaynak	3
lvf_luf_kestirme · tf_yaz	1 · 1
meallow_yaz · _deger	1 · 1
msetflg · mstf_yukle	1 · 1
bellek_yaz · yaz_veri	2 · 2
yaz_adres_arti2 · veri_mr	1 · 2
D2 alanları	6

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
```

E · yürütme

E ilerleme

E'deki buyruğun bu çevrimde gerçekten çalışıp çalışmadığı.

▶ girdi		◀ çıktı	
geçerli	1	e_ilerle	1
dur_exe	1		
dondur	1		

Nasıl çalışır

```
e_ilerle = geçerli ∧ ¬dur_exe ∧ ¬dondur
E'deki bütün yazma izinleri ve ALU valid'leri
bununla VE'lenir
```

d kaynağı muxu

ALU'nun d portunu seçer.

▶ girdi		◀ çıktı	
d_kaynak (k)	2	d	32
MR[d_mr]	32		
imm_hi	32		
imm_shift	32		
okunan	32		

Nasıl çalışır

```
d = case (d_kaynak)
    00 D_MR:     MR[d_mr]
    01 D_IMM_HI: imm_hi
    10 D_SHIFT:  imm_shift
    11 D_BELLEK: okunan
```

Koşul değerlendirici

Koşullu yazmalar ve MTESTTF için CNDF'yi görünen bayraklarla değerlendirir.

▶ girdi		◀ çıktı	
kosul (k)	4	kosul_tuttu	1
gorunen_5	5		

Nasıl çalışır

```
0000 ZF=0       0001 ZF=1
0010 ZF=0∧NF=0  0011 NF=0
0100 NF=1       0101 ZF=1∨NF=1
1010 TF=1       1011 TF=0
1100 LUF=1      1101 LVF=1
1110, 1111 her zaman · 0110–1001 ayrılmış
```

ZF/NF birimi

Buyruğun ZF ve NF değerlerini doğru kuralla üretir.

▶ girdi		◀ çıktı	
zn_kaynak (k)	3	zf_e · nf_e	1 · 1
mstf_out[3:2]	2		
zf/nf_karsilastirma	1 · 1		
e_veri1	32		

Nasıl çalışır

```
case (zn_kaynak)
 1 TAMSAYI:    {zf, nf} = mstf_out[3:2]
 2 KARŞILAŞTIRMA: {zf, nf} = {zf_karş, nf_karş}
 3 YAZILAN_FLOAT: zf = (e_veri1[30:23] == 0)
                  nf = e_veri1[31] ∧ ¬zf
 4 YAZILAN_TAMSAYI: zf = (e_veri1 == 0)
                    nf = e_veri1[31]
```

W'ye veri ve adres

W'de belleğe yazılacak veriyi ve adresi hazırlar.

▶ girdi		◀ çıktı	
yaz_veri (k, seçim)	2	w_veri	32
MR[veri_mr]	32	w_adr	16
MAR0 · MAR1	16 · 16	w_bellek_yaz	2
mar_hedef	1	w_gecerli · w_pc	1 · 12
gorunen_32	32		
okunan	32		
ea	16		
yaz_adres_arti2	1		
bellek_yaz	2		
e_ilerle · pc	1 · 12		

Nasıl çalışır

```
w_veri = case (yaz_veri)
    0 VERI_MR:     MR[veri_mr]
    1 VERI_MAR:    {16'h0, mar_hedef ? MAR1 : MAR0}
    2 VERI_MSTF:   gorunen_32
    3 VERI_BELLEK: okunan        // MMOVD32
w_adr = yaz_adres_arti2 ? ea + 2 : ea
w_bellek_yaz = bellek_yaz, w_gecerli = e_ilerle
```

MR okuma muxları

Paketteki yazmaç numaralarıyla MR0–MR3'ten gerekli değerleri seçer.

▶ girdi		◀ çıktı	
MR0–MR3	4 × 32	a · b · c	32 · 32 · 32
a_mr · b_mr · c_mr	2 · 2 · 2	MR[d_mr]	32
d_mr	2	MR[exe_kaynak_mr]	32
exe_kaynak_mr · _2	2 · 2	e_veri2	32
veri_mr	2	MR[veri_mr]	32
exe_hedef	2	MR[exe_hedef]	32

Nasıl çalışır

```
8 adet 4:1 × 32 mux: çıkış = MR[seçim]
a = MR[a_mr], b = MR[b_mr], c = MR[c_mr]
e_veri2 = MR[exe_kaynak_mr_2]   // MSWAPF
```

ALU kontrol sözcükleri

Paketin alu alanını ALU.sv'nin kontrol girişlerine dağıtır, dinamik bitleri ekler.

▶ girdi		◀ çıktı	
alu (k)	42	topla_cntrl · carpma_cntrl	5 · 5
e_ilerle	1	tamsayi_cntrl · kestirme_cntrl	3 · 1
RND32	1	karsilastirma_cntrl	7
gorunen_5	5	donusum_cntrl	5
		mux_1 … mux_10	10 × 2
		mstf_in	5

Nasıl çalışır

```
topla_cntrl  = {topla_hedef, RND32,
                topla_kullan ∧ e_ilerle, topla_cikarma}
carpma_cntrl = {carp_hedef, RND32,
                carp_kullan ∧ e_ilerle, 1'b0}
tamsayi_cntrl = tamsayi_islem
kestirme_cntrl = kestirme_islem
karsilastirma_cntrl = {karsilastirma_islem, _cndf}
donusum_cntrl = {RND32, donusum_islem}
mux_i = alu.mux[i]
mstf_in = gorunen_5   // {TF, ZF, NF, LUF, LVF}
```

E yazma izinleri

E'nin MR, MAR ve MSTF yazmalarına izin verir.

▶ girdi		◀ çıktı	
e_ilerle	1	e_yaz1 · e_yaz2	1 · 1
kosul_tuttu	1	mar_yaz	1
exe_yaz · exe_yaz_2	1 · 1	mstf_izin	6
exe_kosullu	1		
mar_yukle	1		
zn_kaynak	3		
tf_yaz · meallow_yaz	1 · 1		
msetflg · mstf_yukle	1 · 1		
lvf_luf_kestirme	1		

Nasıl çalışır

```
k_ok   = ¬exe_kosullu ∨ kosul_tuttu
e_yaz1 = e_ilerle ∧ exe_yaz   ∧ k_ok
e_yaz2 = e_ilerle ∧ exe_yaz_2 ∧ k_ok
mar_yaz = e_ilerle ∧ mar_yukle
mstf_izin = e_ilerle ∧ {zn_kaynak ≠ 0, tf_yaz,
   meallow_yaz, msetflg, mstf_yukle, lvf_luf_kestirme}
```

MAR yükleme

MAR'a E'de yüklenecek değeri seçer.

▶ girdi		◀ çıktı	
mar_kaynak (k)	2	mar_veri	16
mar_imm · mar_mr	16 · 16	mar_sec	1
okunan[15:0]	16	mar_en	1
mar_hedef (k)	1		
mar_yaz	1		

Nasıl çalışır

```
mar_veri = case (mar_kaynak)
    0 MAR_IMM:          mar_imm
    1 MAR_MR_ARTI_IMM:  mar_mr
    2 MAR_BELLEK:       okunan[15:0]
mar_sec = mar_hedef,  mar_en = mar_yaz
```

Anlık genişletme

Buyruktaki 16 bitlik değeri ve kaydırma miktarını 32 / 16 bitlik işlenenlere çevirir.

▶ girdi		◀ çıktı	
m	16	imm_hi	32
shift	6	imm_shift	32
MR[exe_hedef][31:16]	16	imm_lo	32
MR[veri_mr][15:0]	16	mar_imm	16
		mar_mr	16

Nasıl çalışır

```
imm_hi    = {m, 16'h0}          // #16FHi, MMOVIZ
imm_shift = {26'b0, shift}      // #SHIFT
imm_lo    = {MR[exe_hedef][31:16], m}  // MMOVXI
mar_imm   = m                   // MMOVI16
mar_mr    = MR[veri_mr][15:0] + m  // 16 bit
```

ALU, E kısmı

Birim girişlerini seçer, kombinasyonel birimleri çalıştırır, toplama/çarpmanın K1'ini yapar.

▶ girdi		◀ çıktı	
a · b · c · d	4 × 32	result_tamsayi	32
topla_cntrl · carpma_cntrl	5 · 5	mstf_out	5
tamsayi_cntrl · kestirme_cntrl	3 · 1	result_kestirme	32
karsilastirma_cntrl	7	lvf/luf_kestirme	1 · 1
donusum_cntrl	5	result_karsilastirma	32
mux_1 … mux_10	10 × 2	zf/nf_karsilastirma	1 · 1
mstf_in	5	result_donusum	32
		K (topla) D · K (çarp) D	70 · 65

Nasıl çalışır

```
mux_i: 00 a · 01 b · 10 c · 11 d → birim girişi
tamsayi_islem: 3 bit kodla 8 işlem (+ − & | ^ << >> >>>)
   ZF = (sonuç == 0), NF = sonuç[31] → mstf_out[3:2]
fpu_kestirme: 1/x ya da 1/√x, estimate_lut.hex
fpu_karsilastirma: CMP, MAX, MIN, ABS, NEG
   NEG: koşul = CNDF(cndf, mstf_in) → −x ya da x
fpu_donusum: 11 dönüşüm, RND32 yalnız int → float
topla_k1: sınıflandır, işaret, büyük üs, hizala
carp_k1: sınıflandır, işaret XOR, üs topla, 24×24
K1 sonucu kenarda ALU içindeki K yazmacına girer
```

E sonuç muxu

MR'ye E'de yazılacak değeri seçer.

▶ girdi		◀ çıktı	
exe_kaynak (k)	3	e_veri1	32
result_tamsayi · _kestirme	32 · 32		
result_karsilastirma · _donusum	32 · 32		
okunan	32		
imm_hi · imm_lo	32 · 32		
MR[exe_kaynak_mr]	32		

Nasıl çalışır

```
e_veri1 = case (exe_kaynak)
  0 result_tamsayi     1 result_kestirme
  2 result_karsilastirma  3 result_donusum
  4 okunan  5 imm_hi  6 imm_lo  7 MR[exe_kaynak_mr]
e_veri1 → MR[exe_hedef], e_veri2 → MR[exe_hedef_2]
```

MSTF, E güncellemesi

E'nin MSTF'ye yazacağı alanları ve hangi alanların yazılacağını (maske) üretir.

▶ girdi		◀ çıktı	
zf_e · nf_e	1 · 1	e_mstf_veri	19
kosul_tuttu	1	e_mstf_maske	19
meallow_deger	1		
m	16		
okunan	32		
lvf/luf_kestirme	1 · 1		
mstf_izin	6		

Nasıl çalışır

```
alan sırası: {RPC 12, MEALLOW, RND32, TF, ZF, NF, LUF, LVF}
zn:       ZF, NF <= zf_e, nf_e
tf:       TF <= kosul_tuttu
meallow: MEALLOW <= meallow_deger
msetflg: FLAG = m[15:8], VALUE = m[7:0]
  FLAG bit 0 1 2 3 6 7 → LVF LUF NF ZF TF RND32
  FLAG[i] = 1 olan alan <= VALUE[i]
yükleme: RPC, RND32, TF, ZF, NF, LUF, LVF <= okunan
  (MEALLOW korunur, maskesi 0)
kestirim: LVF/LUF <= 1 (yalnız lvf/luf_kestirme = 1)
maske = yazılan alanlar
```

E/W · 63 FF + ALU K (135 FF)

W'deki buyruğun bellek yazması; K yazmaçları ALU içindedir.

▶ girdi

w_gecerli · w_pc	1 · 12
w_bellek_yaz	2
w_adr	16
w_veri	32
en_EW · balon_EW	1 · 1
K (topla) D	70
K (çarp) D	65
clk · rst_n	

◀ çıktı

geçerli	1
pc *	12
bellek_yaz	2
yaz_adr	16
yaz_veri	32
K topla: valid · hedef · rnd	1 · 2 · 1
işaret · üs · cik · kod	1 · 8 · 1 · 2
anlamlı_a · anlamlı_b	27 · 27
K çarp: valid · hedef · rnd	1 · 2 · 1
işaret · üs · kod	1 · 10 · 2
çarpım	48

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
dur_exe ise balon_EW = 1
K yazmaçları izinsiz, her kenarda
```

Aşama W: geri yazma

Toplama ve çarpmanın K2'si tamamlanır, sonuç çevrimin sonunda MR'ye yazılır; LVF/LUF MSTF'ye işlenir. Bellek yazması bu çevrimde RAM portundan yapılır. dur_exe kararı da burada, W'deki K2'nin hedefleri ile E'nin okuduğu MR'ler karşılaştırılarak verilir.

boru hattı yazmacı (FF) · mimari durum (FF) · kombinasyonel blok · BRAM · ALU · önerilen arayüz (mevcut modülden farklı) · saat kenarı · ▶ girdi · ◀ çıktı · sağdaki sayı = bit

Dışarıdan gelen sinyaller

sinyal	bit	kaynak
e_yaz1/2, exe_hedef/_2, e_veri1/2	1 · 2 · 32 (× 2)	E sonuç muxu, E yazma izinleri
mstf_q	19	MSTF
geçerli, okunan_mr	1 · 4	R2/E
dondur	1	görev denetçisi

Dışarı giden sinyaller

sinyal	bit	hedef
mr_en, mr_veri	4 · 4 × 32	MR yazmaçları
w_lvf_set, w_luf_set	1 · 1	MSTF
dur_exe	1	boru hattı denetimi, D2, R1, E
gorunen_5, gorunen_32	5 · 32	D2, E
w_blok, w_adr, w_wr_en, w_din	2 · 10 · 2 · 32	R1 · RAM port seçici, dur_r1

E/W · 63 FF + ALU K (135 FF)

▶ girdi

E'den alanlar	63
K (topla) D · K (çarp) D	70 · 65
en_EW · balon_EW	1 · 1
clk · rst_n	

◀ çıktı

geçerli	1
pc *	12
bellek_yaz	2
yaz_adr	16
yaz_veri	32
K (topla) Q	70
K (çarp) Q	65

Nasıl çalışır

```
saat kenarında:
  balon ise geçerli <= 0
  değilse en ise Q <= D
  en = 0 ise Q korunur
```

W · geri yazma

ALU, W kısmı (K2)

K yazmacındaki ara sonucu normalleştirir, yuvarlar ve paketler.

▶ girdi

K (topla) Q	70
K (çarp) Q	65

◀ çıktı

result_topla	32
lvf_topla · luf_topla	1 · 1
valid_topla · hedef_mr_out_topla	1 · 2
result_carp	32
lvf_carp · luf_carp	1 · 1
valid_carp · hedef_mr_out_carp	1 · 2

Nasıl çalışır

```
topla_k2: topla ya da çıkar; taşmada sağa,
   baştaki sıfırlar kadar sola kaydır
carp_k2: 48 bit çarpımı normalleştir, G R S
pack: RND32 ile yuvarla; NaN = 0x7FC00000,
   taşma → LVF, alttan taşma → LUF (sonuç 0)
lvf/luf = pack ∧ valid,  valid/hedef = K'dan
```

Görünen MSTF

Bu çevrimde K2'nin kurduğu LVF/LUF'u da gösteren MSTF görünümü.

▶ girdi

mstf_q	19
lvf_topla · luf_topla	1 · 1
lvf_carp · luf_carp	1 · 1

◀ çıktı

gorunen_5	5
gorunen_32	32

Nasıl çalışır

```
LVF' = LVF ∨ lvf_topla ∨ lvf_carp
LUF' = LUF ∨ luf_topla ∨ luf_carp
gorunen_5  = {TF, ZF, NF, LUF', LVF'}
gorunen_32 = {8'h0, RPC, MEALLOW, 0, RND32,
              00, TF, 00, ZF, NF, LUF', LVF'}
kullanan: D2 dal, E koşul, ALU mstf_in, VERI_MSTF
```

dur_exe birimi

E'deki buyruk, W'de K2'nin yazacağı bir MR'yi okuyorsa beklemeyi başlatır.

▶ girdi

geçerli (R2/E)	1
okunan_mr (k)	4
valid_topla · hedef_mr_out_topla	1 · 2
valid_carp · hedef_mr_out_carp	1 · 2

◀ çıktı

dur_exe	1

Nasıl çalışır

```
k2_maske = (valid_topla ? 1 << hedef_t : 0)
         | (valid_carp  ? 1 << hedef_c : 0)
dur_exe  = geçerli ∧ |(okunan_mr & k2_maske)
kombinasyonel: aynı çevrimde etkili
```

MSTF, W güncellemesi

K2'nin taşma bayraklarını MSTF'ye yapışkan olarak işler.

▶ girdi

lvf_topla · luf_topla	1 · 1
lvf_carp · luf_carp	1 · 1
dondur	1

◀ çıktı

w_lvf_set · w_luf_set	1 · 1

Nasıl çalışır

```
w_lvf_set = (lvf_topla ∨ lvf_carp) ∧ ¬dondur
w_luf_set = (luf_topla ∨ luf_carp) ∧ ¬dondur
MSTF'de: LVF <= E maskesi ? E verisi : LVF ∨ w_lvf_set
```

İz çıkışı (yalnız tezgâh)

Golden modelin izine karşılık gelen satırı her çevrim basar.

▶ girdi

R2/E pc · kelime	12 · 32
MPC	12
MR0–MR3	4 × 32
MAR0 · MAR1	16 · 16
MSTF	32
E/W geçerli · bellek_yaz	1 · 2
E/W yaz_adr · yaz_veri	16 · 32

◀ çıktı

iz satırı	metin

Nasıl çalışır

```
cycle exe_pc exe_op mpc mr0..mr3 mar0 mar1 mstf
      wr_addr wr_data wr_w   (trace_format v1)
dur_exe'de E satırı tekrarlanır
```

MR yazma birimi

Dört yazma kaynağını MR0–MR3'ün izin ve verilerine çevirir.

▶ girdi

e_yaz1 · exe_hedef · e_veri1	1 · 2 · 32
e_yaz2 · exe_hedef_2 · e_veri2	1 · 2 · 32
valid_topla · hedef_t · result_topla	1 · 2 · 32
valid_carp · hedef_c · result_carp	1 · 2 · 32
dondur	1

◀ çıktı

mr_en	4
mr_veri	4 × 32

Nasıl çalışır

```
her x = 0..3 için:
  e1 = e_yaz1 ∧ (exe_hedef == x)
  e2 = e_yaz2 ∧ (exe_hedef_2 == x)
  t  = valid_topla ∧ (hedef_t == x) ∧ ¬dondur
  c  = valid_carp  ∧ (hedef_c == x) ∧ ¬dondur
  mr_en[x]   = e1 ∨ e2 ∨ t ∨ c
  mr_veri[x] = e1 ? e_veri1 : e2 ? e_veri2
             : t  ? result_topla : result_carp
öncelik: E > topla K2 > çarp K2
```

RAM yazma

E/W'deki bellek yazmasını blok, adres ve izin olarak RAM portuna verir.

▶ girdi

geçerli	1
bellek_yaz	2
yaz_adr	16
yaz_veri	32

◀ çıktı

w_blok	2
w_adr	10
w_wr_en	2
w_din	32

Nasıl çalışır

```
w_blok  = çöz(yaz_adr)   // L1, L2, C2M
w_adr   = yaz_adr − taban; 32 bitte w_adr[0] = 0
w_wr_en = geçerli ? bellek_yaz : 00
w_din   = yaz_veri
yazma, R1'deki port seçiciden çevrim sonunda yapılır
```

Ortak: durum yazmaçları, görev denetimi ve boru hattı denetimi

Birden fazla aşamanın okuduğu ya da yazdığı yazmaçlar ve bütün boru hattı yazmaçlarının izin (en) ve balon sinyalleri. Sarı kutular önerilen arayüzdür; mevcut modüllerin portlarından farklıdır.

boru hattı yazmacı (FF) · mimari durum (FF) · kombinasyonel blok · BRAM · ALU · önerilen arayüz (mevcut modülden farklı) · saat kenarı · ▶ girdi · ◀ çıktı · sağdaki sayı = bit

Durum ve denetim

Boru hattı denetimi

Beklemelere, görev olaylarına ve resete göre her boru hattı yazmacının yükleme izni ve balon girişini üretir.

▶ girdi		◀ çıktı	
dur_exe · dur_r1	1 · 1	en_getir_pc · en_F1F2 · en_F2D1	1 · 1 · 1
on_temizle	1	en_D1D2 · en_D2R1	1 · 1
dondur · tumu_temizle	1 · 1	en_R1R2 · en_R2E · en_EW	1 · 1 · 1
getir_izin	1	balon_F1F2 · balon_F2D1	1 · 1
		balon_D1D2 · balon_D2R1	1 · 1
		balon_R1R2 · balon_R2E · balon_EW	1 · 1 · 1

Nasıl çalışır

```
on = ¬(dur_exe ∨ dur_r1 ∨ dondur)
en_getir_pc = en_F1F2 = en_F2D1 = en_D1D2 = en_D2R1 = on
en_R1R2 = en_R2E = ¬(dur_exe ∨ dondur)
en_EW   = ¬dondur
balon_F1F2 = on_temizle ∨ tumu_temizle ∨ ¬getir_izin
balon_F2D1 = balon_D1D2 = balon_D2R1
           = on_temizle ∨ tumu_temizle
balon_R1R2 = (dur_r1 ∧ ¬dur_exe) ∨ tumu_temizle
balon_R2E  = tumu_temizle
balon_EW   = dur_exe ∨ tumu_temizle
```

MAR0, MAR1 · 32 FF

İki adres yazmacı: D2'de artış, E'de yükleme.

▶ girdi		◀ çıktı	
mar_artis_en · _sec	1 · 1	MAR0 · MAR1	16 · 16
mar_artis_deger	16		
mar_en · mar_sec	1 · 1		
mar_veri	16		
clk · rst_n			

Nasıl çalışır

```
her x = 0, 1 için:
  artış x'e ise:  MARx <= mar_artis_deger
  yoksa yükleme x'e ise: MARx <= mar_veri
  ikisi aynı MAR'a düşerse artış kazanır (TRM s.579)
```

cla_gorev_denetci

Hangi görevin ne zaman başlayıp biteceğine karar verir (mevcut modül, portları aynen).

▶ girdi		◀ çıktı	
bekleyen	8	gorev_basla · gorev_no	1 · 3
softreset · hardreset	1 · 1	gorev_bitti	1
d2_gecerli	1	pie_kesme	8
d2_mstop · d2_gecersiz	1 · 1	getir_izin · on_temizle	1 · 1
clk · rst_n		tumu_temizle · dondur	1 · 1
		calisiyor · aktif_gorev	1 · 3

Nasıl çalışır

```
BOŞTA → (bekleyen ≠ 0) → HAKEM1 → HAKEM2
HAKEM2: bekleyen ≠ 0 ise gorev_basla = 1,
  gorev_no = en küçük numaralı bekleyen → CALIŞIYOR
CALIŞIYOR: getir_izin = 1
  d2_gecerli ∧ d2_mstop: gorev_bitti, pie_kesme,
    on_temizle → HAKEM1   (yeni F1 3 çevrim sonra)
  d2_gecerli ∧ d2_gecersiz: pie_kesme, dondur → DONDU
soft/hardreset: → BOŞTA, tumu_temizle
```

MPC · 12 FF

D2'deki buyruğun adresi (izde görünen program sayacı).

▶ girdi		◀ çıktı	
gorev_basla · mvect_pc	1 · 12	MPC	12
mpc_en · mpc_deger	1 · 12		
clk · rst_n			

Nasıl çalışır

```
gorev_basla: MPC <= mvect_pc
mpc_en:      MPC <= mpc_deger
değilse korunur
```

MR0–MR3 · 128 FF

Dört çalışma yazmacı.

▶ girdi		◀ çıktı	
mr_en	4	MR0–MR3	4 × 32
mr_veri	4 × 32		
clk · rst_n			

Nasıl çalışır

```
her x: mr_en[x] ise MRx <= mr_veri[x]
```

cla_ctrl_regs

CPU'nun görev yazmaçları: bekleyen, taşma, izin ve çalışan görev.

▶ girdi		◀ çıktı	
cpu_wr_en · cpu_reg_addr	1 · 3	cpu_rd_data	16
cpu_wr_data	16	bekleyen	8
hw_task_trig	8	mirun	8
gorev_basla · gorev_no	1 · 3	softreset · hardreset	1 · 1
gorev_bitti	1		
clk · rst_n			

Nasıl çalışır

```
bekleyen = MIFR & MIER
CPU yazma: 2 MICLR: MIFR &= ~veri  3 MICLROVF: MIOVF &= ~veri
   4 MIFRC: MIFR |= veri   5 MIER = veri   7 MCTL: reset darbesi
hw_task_trig[i]: MIFR[i] kuruluysa MIOVF[i] = 1
                 değilse MIFR[i] = 1
gorev_basla: MIFR[no] = 0, MIRUN[no] = 1
gorev_bitti: MIRUN = 0
okuma: 0 MIFR, 1 MIOVF, 5 MIER, 6 MIRUN
```

cla_mvect_regs

Sekiz görevin başlangıç adresleri (mevcut modül).

▶ girdi		◀ çıktı	
cpu_wr_en · cpu_wr_task_id	1 · 3	cpu_rd_data	16
cpu_wr_data	16	target_pc_out	16
active_task_id (= gorev_no)	3		
clk · rst_n			

Nasıl çalışır

```
cpu_wr_en: MVECT[cpu_wr_task_id] <= cpu_wr_data
cpu_rd_data   = MVECT[cpu_wr_task_id]
target_pc_out = MVECT[active_task_id]
mvect_pc = target_pc_out[11:0]
```

MSTF · 19 FF

Durum bayrakları ve dönüş adresi.

▶ girdi		◀ çıktı	
e_mstf_veri · e_mstf_maske	19 · 19	mstf_q	19
w_lvf_set · w_luf_set	1 · 1	RND32	1
rpc_en · rpc_deger	1 · 12	RPC	12
clk · rst_n			

Nasıl çalışır

```
alan sırası: {RPC 12, MEALLOW, RND32, TF, ZF, NF, LUF, LVF}
her alan f: maske_E[f] ise f <= veri_E[f]
  değilse LVF <= LVF ∨ w_lvf_set, LUF <= LUF ∨ w_luf_set
RPC: maske_E ise veri_E, değilse rpc_en ise rpc_deger
öncelik: E > K2 (W) > D2
```